

CS 452/652 Winter 2025 - Lecture 3

Polling Loops, Kernel, Tasks

Jan 14, 2025 [prev](#) [next](#)

Concurrency

- manage independent activities or concerns
- conceptual reasons: software structure, runtime management
- practical reasons: parallel hardware
- asynchronous (unordered) execution, non-determinism
 - realtime: priority & preemption

Polling Patterns

- trivial polling loop

```
for (;;) {  
    if (c1) a1;  
    if (c2) a2;  
    ...  
}
```

→ worst-case latency: sum of all actions

- higher-frequency input/condition 'c1'

```
for (;;) {  
    if (c1) a1;  
    if (c2) a2;  
    if (c1) a1;  
    if (c3) a3;  
    ...  
}
```

→ worst-case latency: maximum of all actions

- latency still too high? break up action into parts

```
for (;;) {  
    if (c1) a1;  
    if (c2) { a2.1; half-done = TRUE; }  
    if (c1) a1;  
    if (half-done) { a2.2; half-done = FALSE; }  
    ...  
}
```

→ quickly gets complicated and no runtime flexibility

- A0 simple scenario: Ch2 (train) important, Ch1 faster

Multi-Tasking

- abstraction for concurrency: natural way of thinking about complicated problems

- asynchronous system implemented via synchronous control flow per task
- runtime/kernel: creates and manages tasks (scheduling and priority)
- automatic suspend/resume facilitated by stack and context switching
- aside: why are we using stacks?
 - dynamic variable allocation, especially with threads; recursion

Kernel

- uninterruptible (thus predictable) system manager
 - hardware privilege level → "dangerous" operations possible and permitted
 - central functionality: memory, threading, communication
 - "kernel level" vs. "user level" → hardware has different names
 - CPU executes either kernel or task (with transitions) or interrupt?
- microkernel → device driver logic outside kernel
- task communication → message passing
 - no shared memory; simulated protection
- exceptions: entry/exit between kernel & user-level
 - software interrupt → system call
 - hardware interrupt → from device

Generic Kernel Loop

```
void kmain() {
    initialize(); // includes starting the first user task
    for (;;) {
        currtask = schedule();
        request = activate(currtask);
        handle(request);
    }
}
```

Task State

- management state: Ready, Active, Blocked, etc.
- management information: parent task
 - normally used for group scheduling, termination
- execution state
 - high-level language: line, variables (automatic, static, global, heap)
 - machine: program counter, stack pointer, other registers, stack content
 - data primarily on stack (no heap)
- *task descriptor*: in-kernel data structure holding task state
 - queues for scheduling and blocking

Memory Management

- C/C++ provide very good control over memory layout of structured data
 - use pointer casting to switch between unstructured and structured view (or union declaration)
 - if necessary, [GCC extensions](#) can be used to increase control over memory layout (packed, aligned, etc.)
- techniques for dynamic data structures without heap
 - intrusive linkage, i.e., embed next (and/or other) pointers in data structure
 - slab allocation: dedicated memory region (array of objects) for each data type
 - store freed objects in free list, use intrusive overlay (cast/union) to manage
 - allocate first from free list, then from slab

- task-shared memory: code, readonly data
- task-private memory on stack
 - ok to use static global for singleton tasks with subroutines
 - ok to use static global for boot-time write-once configuration constants
- do not share other memory between tasks!